TASKFLOW

Task Management Web Application

MINOR PROJECT REPORT
Technology Stack: HTML5 · CSS3 · Vanilla JavaScript (ES6+)
Project Type: Academic Submission
Date: October 2026

Prepared for academic minor project submission

Table of Contents

• 1. Project Overview

• 2. Objectives

• 3. Technology Stack

• 4. System Architecture

• 5. Features & Functionality

• 6. Application Screens

• 7. Data Model

• 8. Implementation Details

• 9. Folder Structure

• 10. Code Metrics

• 11. How to Run

• 12. Keyboard Shortcuts

• 13. Limitations & Future Scope

• 14. Conclusion

TaskFlow – Task Management Web Application Minor Project Report

Field	Details
Project Title	TaskFlow – Task Management Web Application
Project Type	Minor Project (Academic Submission)
Technology Stack	HTML5 · CSS3 · Vanilla JavaScript (ES6+)
Date	October 2026
Repository	GitHub Repository

1. Project Overview

TaskFlow is a fully client-side, single-page task management web application designed to help users create, organize, prioritize, and track their tasks efficiently. The application runs entirely in the browser — no backend server, no database installation, and no build tools are required.

The application features a premium dark-themed user interface with smooth animations, real-time filtering and search, overdue detection, a live statistics dashboard, and persistent data storage using the browser's Local Storage API.

TaskFlow is a pure front-end application — all data is stored in the browser's Local Storage under the key taskflow_tasks_v1. No user data is ever sent to a server.

2. Objectives

The primary objectives of the TaskFlow project are:

Demonstrate core web development skills using HTML5, CSS3, and Vanilla JavaScript without any external frameworks or libraries. Build a fully functional CRUD application — users can Create, Read, Update, and Delete tasks. Implement client-side data persistence using the browser's Local Storage API so tasks survive page refresh. Design a responsive, accessible UI that works seamlessly on desktop, tablet, and mobile devices. Practice clean code

architecture — separation of concerns with distinct modules for state management, rendering, CRUD operations, and event handling. Implement real-world UX patterns such as toast notifications, confirmation dialogs, form validation, debounced search, and keyboard shortcuts.

3. Technology Stack

Technology	Version / Standard	Purpose
HTML5	HTML Living Standard	Semantic page structure, ARIA accessibility attributes, form elements
CSS3	CSS Custom Properties, Flexbox, Grid	Responsive layout, dark/light theming, animations, glassmorphism effects
JavaScript	ES6+ (Vanilla)	Application logic, DOM manipulation, event handling, Local Storage
Google Fonts – Inter	Variable font (300–800 wt)	Modern, legible typography
Browser Local Storage	Web Storage API	Client-side data persistence
SVG Icons	Inline SVG	Crisp, scalable iconography (no icon library dependency)

No npm install, no build step, no framework. The entire app is a single folder of static files — open index.html in any modern browser and it works.

4. System Architecture

TaskFlow follows a single-page application (SPA) pattern with a modular structure implemented entirely in Vanilla JavaScript.

Browser (index.html) style.css\nTheme · Layout · Animations script.js\nApplication Core AppState\nGlobal State Object Local Storage\ntaskflow_tasks_v1 DOM Renderer\nbuildTaskCard · renderTaskList Event Listeners\nsetupEventListeners CRUD Operations\nsaveTask · deleteTask · toggleStatus Filters & Sort\ngetFilteredTasks View Router\nswitchView · VIEW_META State Management A single global AppState object holds all runtime state:

```
const AppState = {
```

tasks: [], // Array of all task objects currentView: 'dashboard', taskToDelete: null, // ID queued for deletion editingTaskId: null, // ID of task being edited filters: { status: 'all', priority: 'all', sort: 'createdDate-desc', search: '',

```
},
};
```

View System The application uses a CSS class toggle pattern to show/hide views. Five <section> elements represent the five views (dashboard, tasks, today, overdue, completed). Only the section with the .active class is visible at any time. The switchView() function manages transitions.

5. Features & Functionality

5.1 Task CRUD Operations

Operation	How It Works
Create	Click "New Task" or press Ctrl+N → fill in the modal form → click "Save Task"
Read	Tasks are displayed as cards across all views; dashboard shows a live summary
Update	Click the pencil icon on any task card → edit fields → click "Update Task"

Operation	How It Works
Delete	Click the trash icon → confirm in the dialog → task is permanently removed
Toggle Status	Click the checkbox on any task card to toggle between Pending ↔ Completed

5.2 Dashboard & Analytics

The Dashboard view provides at-a-glance task analytics:

Total Tasks — count of all tasks in storage Pending — tasks that are not yet completed Completed — tasks marked as done Overdue — pending tasks past their due date Overall Progress Bar — animated bar showing (completed / total) × 100% Recent Tasks — the 5 most recently created tasks All stat numbers animate from their previous value to the new value using a smooth counting animation implemented with setInterval.

5.3 Filtering, Searching & Sorting

Available on the All Tasks view:

Control	Options
Status Filter	All · Pending · Completed · Overdue
Priority Filter	All Priorities · High · Medium · Low
Sort By	Newest First · Oldest First · Due Date (Earliest/Latest) · Priority (High→Low / Low→High) · Title (A–Z / Z–A)
Global Search	Debounced live search across task title and description

5.4 Visual Priority System

Each task card has a colored left-border stripe and a priority badge to communicate urgency at a glance:

Priority	Color	Badge
■ High	Red (#ef4444)	Red badge
■ Medium	Amber (#f59e0b)	Amber badge
■ Low	Green (#22c55e)	Green badge

5.5 Due Date Intelligence

Tasks past their due date are automatically classified as Overdue (red highlight) Tasks due on today's date are highlighted in amber The sidebar badges for "All Tasks" and "Overdue" update in real-time

5.6 Theme Toggle

Dark Mode (default) and Light Mode are supported The selected theme is stored in localStorage under the key taskflow_theme The toggle is in the sidebar footer and persists across sessions

5.7 Toast Notifications

Non-blocking slide-in toast notifications confirm every action:

Action	Toast Type
Task created	■ Success (green)
Task updated	■ Success (green)

Action	Toast Type
Task deleted	■■ Info (purple)
Status toggled	■ / ■■ based on new status
Storage error	■ Error (red)

5.8 Accessibility (ARIA)

The application includes:

Semantic HTML (<main>, <aside>, <nav>, <header>, <article>, <section>) ARIA roles and labels on all interactive elements aria-live="polite" on the toast container role="progressbar" with aria-valuenow on the progress bar aria-modal="true" and aria-labelledby on dialogs aria-current="page" on active navigation links

6. Application Screens

6.1 Dashboard

Dashboard View

The Dashboard displays four stat cards (Total, Pending, Completed, Overdue), an animated progress bar showing overall completion percentage, and a list of the 5 most recently created tasks.

6.2 Task List (All Tasks View)

Task List View

The All Tasks view displays every task with filtering by status/priority, sorting options, and a live search bar. Each task card shows priority badge, status badge, due date, and action buttons.

6.3 Add / Edit Task Modal

Task Form Modal

The modal form allows creating or editing a task. Fields include title (with character counter up to 120), description (up to 500 characters), priority selector, due date picker, and a status toggle. All required fields are validated before saving.

6.4 Delete Confirmation Dialog

Delete Confirmation

When the user clicks the trash icon, a compact confirmation modal appears naming the task, preventing accidental deletion. The user must explicitly click "Delete Task" to confirm.

7. Data Model

Each task is stored as a plain JavaScript object with the following schema:

```
{
"id":          "tf_1728000000000_abc123",
"title":       "Write project report",
"description": "Document the project objectives and implementation.",
"priority":    "high",
"status":      "pending",
"dueDate":     "2026-10-20",
"createdAt":   "2026-10-15T10:00:00.000Z",
"updatedAt":   "2026-10-15T10:00:00.000Z"
}
```

Field	Type	Description
id	string	Unique ID — tf_<timestamp>_<random>

Field	Type	Description
title	string	Task title (2–120 characters, required)
description	string	Optional task description (0–500 characters)
priority	"high" | "medium" | "low"	Task urgency level (required)
status	"pending" | "completed"	Current task status
dueDate	string	Due date in YYYY-MM-DD format (required)
createdAt	string	ISO 8601 timestamp of creation
updatedAt	string	ISO 8601 timestamp of last modification

All tasks are persisted as a JSON array in localStorage.getItem('taskflow_tasks_v1').

8. Implementation Details

8.1 Unique ID Generation

```
function generateId() {
return `tf_${Date.now()}_${Math.random().toString(36).slice(2, 8)}`;
}
```

IDs are prefixed with tf_, followed by a millisecond timestamp for ordering, and a short random base-36 string for collision resistance.

8.2 Overdue Detection

```
function isOverdue(task) {
if (task.status === 'completed') return false;
return task.dueDate && task.dueDate < todayStr();
}
```

A task is overdue if it is not completed and its dueDate is strictly before today's date string. Completed tasks are never overdue regardless of their due date.

8.3 Debounced Search

```
function debounce(fn, delay) {
```

let timer; return (...args) => { clearTimeout(timer); timer = setTimeout(() => fn(...args), delay);

```
};
}
```

The global search input uses a 300ms debounce to avoid triggering a full re-render on every keystroke, improving performance especially with large task lists.

8.4 XSS Prevention

```
function escapeHtml(str) {
const div = document.createElement('div');
```

div.appendChild(document.createTextNode(str)); return div.innerHTML;

```
}
```

All user-supplied text is passed through escapeHtml() before being inserted into the DOM via innerHTML, preventing Cross-Site Scripting (XSS) attacks.

8.5 Animated Stat Numbers

```
function animateNumber(id, target) {
const el = document.getElementById(id);
const start = parseInt(el.textContent) || 0;
const range = target - start;
const steps = 20;
```

let step = 0; const timer = setInterval(() => { step++; el.textContent = Math.round(start + (range * step / steps)); if (step >= steps) { el.textContent = target; clearInterval(timer); }

```
}, 16); // ~60fps
}
```

Stat card numbers animate from their previous value to the new value over ~320ms at ~60fps using linear interpolation.

8.6 CSS Custom Properties for Theming

The entire color system is defined as CSS custom properties on :root (dark) and .light-theme (light), enabling the complete theme toggle with a single class swap:

```
:root {
```

--bg-primary: #0f0f1a; --bg-secondary: #1a1a2e; --accent: #6c63ff; --text-primary: #e2e8f0; /* ... */

```
}
```

.light-theme { --bg-primary: #f8fafc; --bg-secondary: #ffffff; --text-primary: #1e293b; /* ... */

```
}
```

9. Folder Structure

```
TaskFlow_Minor_Project/
■■■ source_code/
■   ■■■ index.html          ← Main HTML file (single page, 384 lines)
■   ■■■ style.css           ← Complete stylesheet (dark/light themes, 30 KB)
■   ■■■ script.js           ← Full application logic (879 lines)
■   ■■■ assets/
■       ■■■ banner.svg      ← Project banner image
■■■ screenshots/
■   ■■■ dashboard.png       ← Dashboard view screenshot
■   ■■■ task_form.png       ← Add/Edit task modal screenshot
■   ■■■ task_list.png       ← All Tasks view screenshot
■   ■■■ Delet_task.png      ← Delete confirmation screenshot
■■■ Project Report (1).pdf  ← Submitted project report (PDF)
■■■ README.md               ← Developer documentation
■■■ GitHub_Repository_Link.txt  ← Deployment links
```

10. Code Metrics

File	Lines of Code	File Size
index.html	383	~19 KB
style.css	1,082	~30 KB
script.js	878	~31 KB
Total	2,343	~80 KB

The entire application weighs under 80 KB of hand-written source code with zero external JavaScript dependencies.

11. How to Run

Prerequisites Any modern web browser: Chrome 90+, Firefox 88+, Edge 90+, Safari 14+ No Node.js, npm, or build tools required Option 1 — Direct Open (Simplest)

Double-click: source_code/index.html Option 2 — VS Code Live Server Install the Live Server extension Right-click index.html → Open with Live Server Option 3 — Python HTTP Server

cd source_code python -m http.server 8080 # Visit: http://localhost:8080 Option 4 — Node.js serve

cd source_code npx serve . # Visit: http://localhost:3000

12. Keyboard Shortcuts

Shortcut	Action
Ctrl + N	Open the New Task modal
Ctrl + F	Focus the global search bar
Escape	Close any open modal or dialog

13. Limitations & Future Scope

Current Limitations

Limitation	Details
Client-side only	No user accounts; data is browser-local and not synced across devices
No backend	Cannot share tasks with other users
Local Storage capacity	Browsers typically limit Local Storage to ~5 MB
No subtasks	Tasks are flat — no support for checklists or subtasks
No file attachments	Cannot attach files or images to tasks
No recurring tasks	Each task has a single due date; no repeat/recurrence support

Future Scope

- ■ User Authentication — Add login/sign-up with Firebase Auth or Supabase
- ■■ Cloud Sync — Store tasks in Firestore or a REST API for cross-device access
- ■ Progressive Web App (PWA) — Add a Service Worker and manifest to enable offline use and home screen installation
- ■■ Tags / Labels — Allow tasks to be tagged with custom categories
- ■ File Attachments — Support attaching images or documents
- ■ Recurring Tasks — Daily, weekly, and monthly task recurrence
- ■ Analytics Dashboard — Charts and graphs for task completion trends
- ■ Browser Notifications — Push reminders for upcoming and overdue tasks
- ■■ Projects / Workspaces — Group related tasks under projects
- ■ Collaboration — Multi-user task assignment and comments

14. Conclusion

TaskFlow demonstrates a complete, production-quality single-page web application built exclusively with native web technologies — HTML5, CSS3, and Vanilla JavaScript. The project covers the full spectrum of front-end development skills:

Structural design with semantic, accessible HTML Visual design with a polished dark-theme UI using CSS Custom Properties, Flexbox, Grid, and CSS animations Application logic implementing the full CRUD lifecycle, state management, event-driven architecture, filtering, sorting, and client-side persistence UX best practices including form validation, confirmation dialogs, toast notifications, debounced search, keyboard shortcuts, and mobile responsiveness The application is lightweight (< 80 KB of source code), dependency-free, and runs instantly in any modern browser without any setup.

Report prepared for academic minor project submission — October 2026. Built with HTML, CSS & JavaScript.